

Week 2 Lab (Task 1) - Cleaning, Preparing & Analysing Data for AI Applications

Data Cleaning (Practical Example)

Data cleaning is a vital process that enhances the accuracy and reliability of analyses and machine learning models.

This introductory notebook will guide you through common data cleaning techniques in Python, demonstrating step-by-step procedures to handle common examples of messy data.

For all future labs, you must first create a 'data' folder at the root folder of your google drive. Place in the provided csv files so the labs all work.

Mount the google drive

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Read in data from the folder

```
data_folder = "drive/MyDrive/data"
```

```
data = pd.read_csv(f'{data_folder}/messy_data.csv')
data
```

	client_id	entity_type	entity_year_established	transaction_id	contract_id	transaction_date	payment_amt	payment_
0	786	Australian Private Company	2002	25555	729	1532455690	40.0	PAYM
1	786	Australian Private Company	2002	16056	729	1522256904	40.0	PAYM
2	786	Australian Private Company	2002	25554	729	1532455690	250.0	PAYM
3	786	Australian Private Company	2002	20593	729	1527530912	\$250.00	pay
4	786	Australian Private Company	2002	8657	729	1511888893	250.0	PAYM
...	...	...	...	...	...	...	...	...
25849	960	Australian Private Company	2001	25440	849	1532369288	80.0	PAYM
25850	960	Australian Private Company	2001	17859	849	1524506909	500.0	PAYM
25851	960	Australian Private Company	2001	6804	849	1509037694	80.0	PAYM
25852	960	Australian Private Company	2001	15791	849	1521997707	500.0	PAYM
25853	960	Australian Private Company	2001	17860	849	1524506909	80.0	PAYM

25854 rows × 8 columns

Next steps: [Generate code with data](#) [New interactive sheet](#)

▼ Data types

The first thing we want to do is explore all the data types. We don't want numbers being stored as strings because then we can't perform mathematical operations on them. The same applies to dates - we can't analyse date columns if they are not in a correct format.

From the output below, you can see that the `payment_amt` column is non-numeric (it is of an `'object'` datatype), despite it appearing to be a numerical value upon first inspection.

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25854 entries, 0 to 25853
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   client_id             25854 non-null  int64  
 1   entity_type           25854 non-null  object  
 2   entity_year_established 25854 non-null  int64  
 3   transaction_id         25854 non-null  int64  
 4   contract_id           25854 non-null  int64  
 5   transaction_date       25854 non-null  int64  
 6   payment_amt           25854 non-null  object  
 7   payment_code          25854 non-null  object  
dtypes: int64(5), object(3)
memory usage: 1.6+ MB
```

The remedy here is often to check the unique values

```
# Attempt to convert values to numeric, non-numeric values will become NaN
numeric_data = pd.to_numeric(data['payment_amt'], errors='coerce')

# Check for NaN values to identify non-numeric entries
non_numeric_indices = pd.isna(numeric_data)

# Display non-numeric values
data.loc[non_numeric_indices, :]
```

	client_id	entity_type	entity_year_established	transaction_id	contract_id	transaction_date	payment_amt	payment
3	786	Australian Private Company	2002	20593	729	1527530912	\$250.00	pa
444	55	Australian Private Company	2016	9662	374	1513530498	\$75.00	pa
991	871	Australian Private Company..	2007	12872	829	1518368902	\$75.00	pa
5546	796	Australian Private Company	2017	597	485	1499710095	\$162.50	PAYI
23761	305	Individual/Sole Trader	2016	10832	1006	1515344901	\$133.33	PAYI

Now remove the dollar signs using regular expressions and convert to numeric form

```
data.loc[non_numeric_indices, 'payment_amt'] = (data.loc[non_numeric_indices, 'payment_amt']
                                                .replace('\$', '', regex=True))

# Convert to numeric because the data are still in string format
data['payment_amt'] = pd.to_numeric(data['payment_amt'])

print(f"Format is now of type -> {data['payment_amt'].dtype}")
```

```
Format is now of type -> float64
<>:2: SyntaxWarning: invalid escape sequence '\$'
<>:2: SyntaxWarning: invalid escape sequence '\$'
/tmp/ipykernel_433/2621630020.py:2: SyntaxWarning: invalid escape sequence '\$'
      .replace('\$', '', regex=True))
```

✖ Inconsistent data

Another thing to be aware of is inconsistent data such as potential mis-spellings or case issues with text (e.g. 'Sydney' vs 'sydney' vs 'SyDneY')

```
def identify_uniques(dataset: pd.DataFrame, text_columns: list[str]):
    for col in text_columns:
        uniques = dataset[col].unique()
        print(f'Unique values for column `{col}`:')
        print(uniques)
        print()

# Call the function
identify_uniques(data, data.select_dtypes(include='object'))
```

```
Unique values for column `entity_type`:
['Australian Private Company' ' Australian Private Company'
 'Australian Private Company..' 'Individual/Sole Trader..'
 'Individual/Sole Trader' 'Family Partnership..' 'Family Partnership'
 'Individual/Sole Trader' 'Family Partnership '
 'Australian Proprietary Company' 'Australian Proprietary Company..'
 'Discretionary Trading Trust..' 'Discretionary Trading Trust'
 'Discretionary Investment Trust' 'Discretionary Investment Trust..'
 'Australian Public Company' 'Australian Public Company..'
 'Other Partnership..' 'Other Partnership' 'Fixed Unit Trust'
 'Fixed Unit Trust..' 'Hybrid Trust']

Unique values for column `payment_code`:
['PAYMENT' 'payment' 'DEFAULT' 'default']
```

Here we see two issues.

1. In the `entity_type` column there are different representations of the same entity type (e.g. 'Australian Private Company', ' Australian Private Company', 'Australian Private Company..')
2. In the `payment_code` column there are case issues (some in uppercase and some in lowercase)

First let's get the strings in a consistent title format

```
data = data.applymap(lambda x: x.title() if type(x) == str else x)
```

```
/tmp/ipykernel_433/953497612.py:1: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  data = data.applymap(lambda x: x.title() if type(x) == str else x)
```

```
# Call the function
identify_uniques(data, data.select_dtypes(include='object'))
```

```
Unique values for column `entity_type`:
['Australian Private Company' ' Australian Private Company'
 'Australian Private Company..' 'Individual/Sole Trader..'
 'Individual/Sole Trader' 'Family Partnership..' 'Family Partnership'
 'Individual/Sole Trader' 'Family Partnership '
 'Australian Proprietary Company' 'Australian Proprietary Company..'
 'Discretionary Trading Trust..' 'Discretionary Trading Trust'
 'Discretionary Investment Trust' 'Discretionary Investment Trust..'
 'Australian Public Company' 'Australian Public Company..'
 'Other Partnership..' 'Other Partnership' 'Fixed Unit Trust'
 'Fixed Unit Trust..' 'Hybrid Trust']

Unique values for column `payment_code`:
['Payment' 'Default']
```

The code `data = data.applymap(lambda x: x.title() if type(x) == str else x)` is used to format all string entries in a pandas DataFrame to title case.

This means that for every string entry in the DataFrame, the first letter of each word is capitalized, and the remaining letters are in lowercase.

The `applymap` function ensures this transformation is applied to each element, and the `lambda` function checks if an element is a string before applying the `title()` method. If an element is not a string, it remains unchanged.

Now let's delete the double periods and replace instances where there are two or more spaces. Lastly, we will strip whitespace at the beginning and end of off the strings

```
# Cleaning up the 'entity_type' column
data['entity_type'] = (data['entity_type']
                      .str.replace('\.\\.', '', regex=True) # Remove two consecutive dots
                      .str.replace(r'\s+', ' ', regex=True) # Replace multiple spaces with a single space
                      .str.strip()) # Trim whitespace from the start and end
```

```
<>:3: SyntaxWarning: invalid escape sequence '\\.'
<>:3: SyntaxWarning: invalid escape sequence '\\.'
/tmp/ipykernel_433/1081215016.py:3: SyntaxWarning: invalid escape sequence '\\.'
      .str.replace('\.\\.', '', regex=True) # Remove two consecutive dots
```

```
data['entity_type'].unique() ## done
```

```
array(['Australian Private Company', 'Individual/Sole Trader',
      'Family Partnership', 'Australian Proprietary Company',
      'Discretionary Trading Trust', 'Discretionary Investment Trust',
      'Australian Public Company', 'Other Partnership',
      'Fixed Unit Trust', 'Hybrid Trust'], dtype=object)
```

▼ Outlier data (+ impossible values)

We also need to investigate the numeric column of `payment_amt` and ensure that the values 'make sense'. A negative payment amount is not possible in this sense so let's filter out all rows where the payment amount is negative

```
print(data.shape)

data.query(expr='payment_amt >= 0', inplace=True)

print(data.shape) # two rows deleted
```

```
(25854, 8)
(25852, 8)
```

▼ Date formatting

The dates are being stored as unix timestamps, which is the number of seconds that have passed since January 1, 1970 (the Unix epoch). This isn't of much use most of the time, so let's change it.

```
data['transaction_date']
```

	transaction_date
0	1532455690
1	1522256904
2	1532455690
3	1527530912
4	1511888893
...	...
25849	1532369288
25850	1524506909
25851	1509037694
25852	1521997707
25853	1524506909

25852 rows × 1 columns

dtype: int64

Convert the unix timestamps to a more interpretable date format

```
data['transaction_date_str'] = pd.to_datetime(data['transaction_date'], unit='s')
data['transaction_date_str'].head()
```

```

transaction_date_str
0    2018-07-24 18:08:10
1    2018-03-28 17:08:24
2    2018-07-24 18:08:10
3    2018-05-28 18:08:32
4    2017-11-28 17:08:13

```

dtype: datetime64[ns]

How could we translate that unix timestamp to Sydney Australia timezone? Hint: Look up pytz.

```

import pytz

# Convert 'transaction_date' to datetime in UTC and then localize to Sydney timezone
sydney_timezone = pytz.timezone('Australia/Sydney')
data['transaction_date_str_syd'] = pd.to_datetime(data['transaction_date'], unit='s', utc=True).dt.tz_convert(sydney_timezone)

# Display the first few rows
print(data['transaction_date_str_syd'].head())

0    2018-07-25 04:08:10+10:00
1    2018-03-29 04:08:24+11:00
2    2018-07-25 04:08:10+10:00
3    2018-05-29 04:08:32+10:00
4    2017-11-29 04:08:13+11:00
Name: transaction_date_str_syd, dtype: datetime64[ns, Australia/Sydney]

```

▼ Duplicated data

From the command below, we see that there are no entire copies of particular rows

```
data[data.duplicated()]
```

client_id	entity_type	entity_year_established	transaction_id	contract_id	transaction_date	payment_amt	payment_code
-----------	-------------	-------------------------	----------------	-------------	------------------	-------------	--------------

Having said this we need to ensure that the `transaction_id` is not duplicated. It would not make sense for there to be multiple transactions with the same ID so let's check this by finding any duplicate `transaction_id` values, seeing *all* the rows in which duplication was involved (`keep=False`) and sorting the values to see the duplicates on top of one another.

```

data[
    data.duplicated(
        subset=['transaction_id'], keep=False
    )
].sort_values(by='transaction_id')

```

	client_id	entity_type	entity_year_established	transaction_id	contract_id	transaction_date	payment_amt	payment_	
	21383	797	Discretionary Investment Trust	2016	239	474	1499191687	433.33	Pay
	21448	797	Australian Private Company	2002	239	474	1499191687	433.33	Pay
	21447	797	Australian Private Company	2002	240	474	1499191687	34.67	Pay
	21382	797	Discretionary Investment Trust	2016	240	474	1499191687	34.67	Pay
	21360	797	Discretionary Investment Trust	2016	322	474	1499364489	433.33	De
	...	...	...	...	...	...	...	...	
	5428	591	Australian Private Company	2015	25485	1103	1532455688	1666.67	De
	5272	591	Australian Private Company	2013	25485	1103	1532455688	1666.67	De
	5271	591	Australian Private Company	2013	25486	1103	1532455688	266.66	De
	5427	591	Australian Private Company	2015	25486	1103	1532455688	266.66	De
	5349	591	Australian Private Company	2007	25486	1103	1532455688	266.66	De

512 rows × 10 columns

Keep only the first of the duplicated values because every cell with the same `transaction_id` has identical values for every column apart from the `entity_type` and `entity_year_established`. Hence we will keep the observation from the most recent of years.

```
indices_to_drop = (data[data.duplicated(subset=['transaction_id'], keep='first')]
                    .sort_values(by='transaction_id')
                    .index)
indices_to_drop
```

```
Index([21448, 21447, 21425, 21426, 21405, 21406, 7120, 7109, 5414, 5492,
      ...,
      5406, 5484, 5344, 5422, 5340, 5418, 5350, 5428, 5427, 5349],
      dtype='int64', length=295)
```

```
data.drop(indices_to_drop, inplace=True)
```

Double check the change was made

```
data[data.duplicated(subset='transaction_id')]
```

	client_id	entity_type	entity_year_established	transaction_id	contract_id	transaction_date	payment_amt	payment_
5338	591	Australian Private Company	2007	24938	1619	1531937359	50.00	Pay
5339	591	Australian Private Company	2007	20613	1103	1527530912	1666.67	Pay
5340	591	Australian Private Company	2007	25320	1619	1532282891	249.99	Pay
5341	591	Australian Private Company	2007	20140	1103	1527012510	266.66	De
5342	591	Australian Private Company	2007	24937	1619	1531937359	20050.04	Pay
...	...	...	...	...	...	...	...	
23677	1262	Australian Private	2016	21564	1439	1528654154	97.50	Pay

Reflection

In this exercise, you worked with a CSV file using pandas and performed basic data cleaning. Now write a reflection of around 5–8 sentences describing:

- What data cleaning steps you performed
- Any issues or challenges you encountered
- What you learned about working with messy or real-world data
- Why data cleaning is important before analysis

23678

1262

Australian Private Company

2016

24192

1584

1531246159

9999.97

Pay

23679

1262

Australian Private Company

2016

24547

1584

1531678160

833.34

Pay

23680

1262

Australian Private Company

2016

23825

1439

1530814127

2500.00

Pay

23681

1262

Australian Private Company

2016

21155

1439

1528222125

0.01

Pay

#Enter your answer here.

295 rows × 10 columns

Reflection

During this lab, I cleaned and prepared a dataset using pandas. I corrected data types, fixed inconsistent text values, removed invalid records, converted timestamps into readable dates, and handled duplicate transaction IDs. The exercise demonstrated how real-world